BLG202E Numerical Methods in Comp. Eng.

Spring 2026 – Term Project

Due: May 17, 2026

By turning in this assignment, I agree by the ITU honor code and declare that all of this is my own work.

Important Notes

- Upload your solutions through Ninova. Projects sent via e-mail and late submissions will not be accepted.
- Write your full name and student ID on every file you submit.
- Cheating is strictly prohibited and will result in a negative grade and disciplinary action. All work must be your own. Teamwork is not allowed.
- All code and reports will be checked for plagiarism. Do not copy text or code from any source.
- If you have any questions, please contact Res. Asst. Enes Erdoğan (erdogane16@itu.edu.tr).

Applications of SVD in Point Cloud Processing

In this project, you will explore applications of Singular Value Decomposition (SVD) in point cloud processing. Briefly, you will use Principal Component Analysis (PCA) to compute geometric properties and use Iterative Closest Point (ICP) to align two point clouds. You are given a set of point cloud data files (some of which are depicted in Figure 1) and you will complete the following tasks:

1. Compute oriented bounding boxes (OBBs) for the floating boxes.
2. Estimate surface normals for the bunny.
3. Compute curvature values for the bunny.
4. Align two bunny point clouds captured from different viewpoints using the Iterative Closest Point (ICP) algorithm.

All of these tasks rely on SVD, either to compute eigenvalues/eigenvectors (via PCA) or to find the optimal rigid transformation (in ICP). You are expected to implement SVD from scratch using the numerical methods covered in the lectures, and to visualize the result of each task separately. Finally, you need to prepare a project report that covers the implementation details and the results.

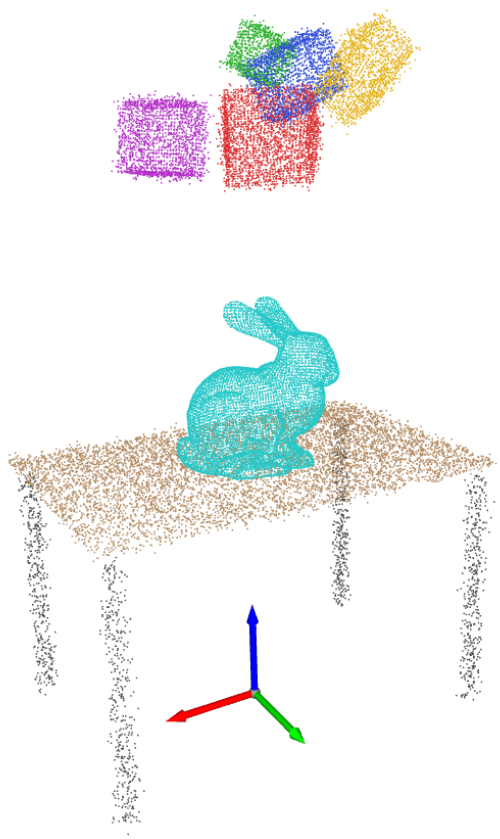

Figure 1: A highly realistic scene with a bunny* on a table and boxes floating in the air.

*This bunny is the famous Stanford bunny model, for more information, you can check its website.

Background

Principal Component Analysis

Principal Component Analysis (PCA) is a statistical method that identifies the orthogonal directions of maximum variance in a dataset. Given a set of data points, PCA computes the covariance matrix Σ and finds its eigenvectors and eigenvalues through eigendecomposition. The eigenvectors, called the *principal components*, form an orthogonal basis that diagonalizes Σ. In other words, the principal components define a rotation into a coordinate system where the data dimensions are uncorrelated and the variance along each successive axis is maximized.

PCA is closely related to the Singular Value Decomposition (SVD): performing SVD on the mean-centered data matrix $X = USV^T$ yields the principal components as the columns of V, with the singular values related to the eigenvalues of Σ by $\lambda_i = s_i^2$.

PCA is commonly used for dimensionality reduction, where data is projected onto the top-k principal components to obtain a lower-dimensional representation that preserves the most variance. However, in the context of point clouds, the principal components of a local neighborhood of points reveal the geometric structure at that location.

For a deeper understanding of PCA, I **strongly** recommend reading the tutorial paper by Shlens [1], which provides solid intuition and mathematical background for PCA and its relation to SVD.

Iterative Closest Point

The Iterative Closest Point (ICP) [2] algorithm is a widely used method for aligning two point clouds of the same object from different views. It iteratively finds the closest points between the two point clouds and computes the rotation matrix and translation vector that minimizes the distance between the matched pairs. The algorithm is guaranteed to converge to a local minimum, and will reach the global minimum if the initial guess is close enough to the true transformation.

The ICP algorithm consists of the following steps:

1. Estimate an initial transformation so that the two point clouds are roughly aligned (there are many ways to do this, but it is not the focus of this project).
2. For each point in the source cloud, find the closest point in the target cloud.
3. Compute the optimal rigid transformation (rotation and translation) that minimizes the distance between the matched pairs using the Kabsch algorithm, which relies on SVD.
4. Apply the transformation to the source cloud and repeat from step 2 until the change in error falls below a chosen threshold or a maximum number of iterations is reached.

Tasks

The first three tasks are related to PCA, and the last task is related to ICP. All tasks use SVD somewhere in the implementation.

Oriented Bounding Box

An *axis-aligned bounding box* (AABB) is the smallest box enclosing a set of points whose edges are parallel to the coordinate axes. While simple to compute, an AABB can be a poor fit for objects that are not aligned with the coordinate axes, resulting in a much larger box than necessary. On the other hand, an *oriented bounding box* (OBB), which is aligned with the principal directions of the object, typically yields a tighter fit as shown in Figure 2.

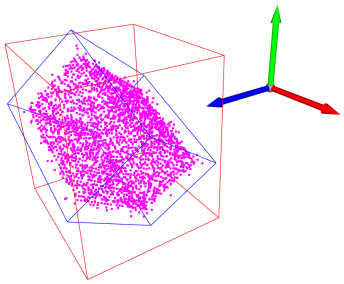

Figure 2: A comparison between an AABB (red) and an OBB (blue) for the point cloud of the purple box.

To compute an OBB, first compute the principal components of the point cloud for each box separately. You need to figure out how to fit an OBB to the point cloud using the principal components. Implement this in your code and visualize your results for all the boxes.

Surface Normals

The surface normal at a point can be understood by imagining a tangent plane fitted to the local surface around that point; the normal vector is simply the vector perpendicular to this plane.

To estimate the surface normal at a point, perform PCA on its local neighborhood (i.e. its k-nearest neighbors). Determine which principal component corresponds to the surface normal and explain why. Then repeat the same process for every point in the point cloud. For this task, only consider the bunny point cloud.

The expected result is shown in Figure 3. Notice that the direction of each normal vector is mapped to a color. Additionally, a line is drawn from each point to indicate the normal direction. Since the point cloud is dense, you may subsample the displayed normal vectors to keep the visualization readable. You are expected to implement a similar visualization in your code.

Note that an eigenvector only defines a line, not a direction, so there is a *sign ambiguity*: the estimated normal could point inward or outward with equal likelihood. There are different ways to resolve this ambiguity, but for simplicity, compute the vector from the

centroid of the entire bunny point cloud to the query point. If the dot product between this vector and the estimated normal is negative, flip the normal so that it points outward.

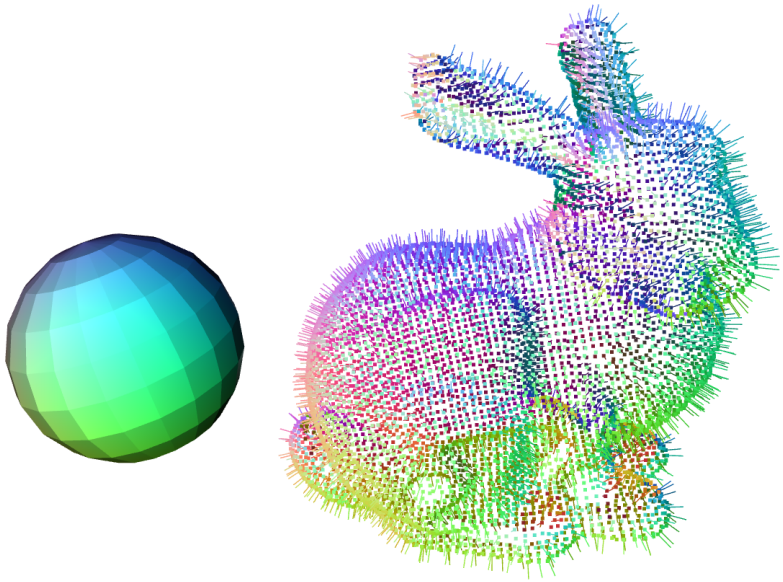

Figure 3: Estimated surface normals of the bunny. The left ball is just for visualization purposes (to show the relation between the direction of the normal vector and the color of the point).

Curvature Values

As in the previous task, perform PCA on the local neighborhood of each point of the bunny. The curvature value is defined as the ratio of the smallest eigenvalue to the sum of all eigenvalues:

$$\sigma = \frac{\lambda_3}{\lambda_1 + \lambda_2 + \lambda_3}$$

Compute this value for every point in the bunny point cloud and visualize the result using a colormap. Interpret the results and discuss the meaning of the curvature values.

Aligning Two Point Clouds

For this task, you are given a pair of bunny point clouds captured from two different viewpoints. Your goal is to align them into a common coordinate system using the Iterative Closest Point (ICP) algorithm. An initial transformation has already been applied, so the two point clouds are *roughly* aligned. Iteratively find closest-point correspondences and compute the optimal transformation using the **Kabsch algorithm**[3]. Print the alignment error after each iteration so that the convergence behavior is visible. For this task, use the point clouds in the `icp` folder.

The expected result is shown in Figure 4.

[3]The Kabsch algorithm finds the optimal rotation matrix and translation vector that minimizes the RMSD between two paired sets of points. It uses SVD of the cross-covariance matrix of the point sets. For more details, check out its wiki page here.

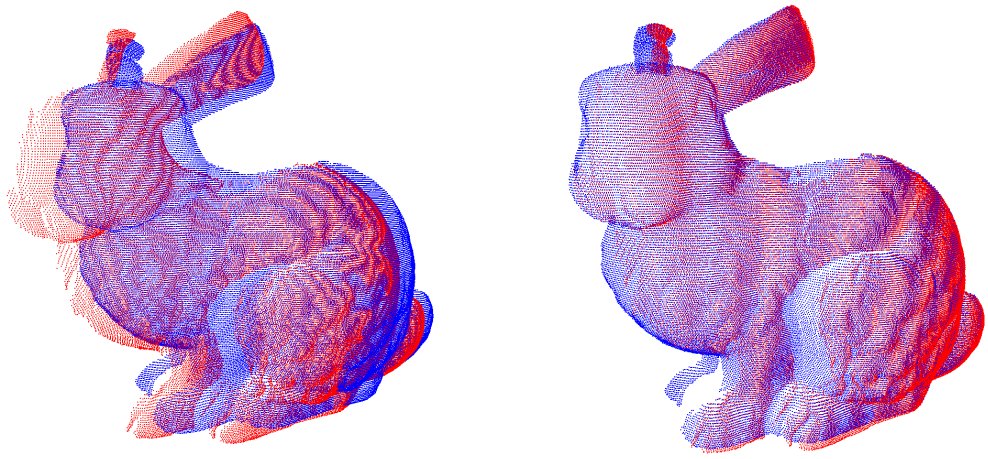

Figure 4: The initial and aligned point clouds of the bunny.

Implementation

You must use Python 3.9–3.12 for this project. You can use the following libraries **only** for the purposes listed below:

- **NumPy** – for basic numerical operations (e.g. matrix storage, multiplication, manipulation).
- **Open3D** – for reading point cloud files and visualizing results. You can also use the `LineSet` class to draw lines in the visualization. However, be aware that you cannot save the `LineSet` objects together with `PointCloud` objects in a single file. So, when saving the result, sample points along the lines at a fine enough resolution and merge them with the point cloud before saving to a single file.
- **SciPy** – *only* for the KD-Tree data structure (`scipy.spatial.KDTree`) to speed up neighborhood queries. You need this for all tasks except the first one, otherwise it will be slow.

Do not use any other external libraries or built-in high-level functions that directly solve the tasks (e.g. `open3d.geometry.OrientedBoundingBox`). In particular, you must implement SVD yourself using the methods you learned in the lectures (do not call library SVD/eigendecomposition routines such as `numpy.linalg.svd` or `numpy.linalg.eig`).

Below is a starter code for reading and visualizing a point cloud file and accessing the points as a NumPy array:

```
import open3d as o3d
import numpy as np

# coordinate frame
axes = o3d.geometry.TriangleMesh.create_coordinate_frame(size=0.2)

pcd = o3d.io.read_point_cloud("data/pca/scene.pcd")
```

```
points = np.asarray(pcd.points)
...
o3d.visualization.draw_geometries([pcd, axes])
```

This will open a visualization window that you can interact with. The Ctrl, Shift, and Alt keys modify how the mouse controls the view. Press `h` for more information. For more details about the visualization, or any other functionalities of Open3D in general, please refer to the Open3D documentation.

If you are using VSCode (or one of its derivatives), you can use this extension for yourself for faster visualization.

Submission

Prepare a project report using the IEEE Conference Template. The report must be at most 5 pages. Properly cite all sources you used.

Submit a separate Python file for each task and a separate Python file where you compare your SVD implementation with NumPy's SVD numerically, along with any additional file for shared functions. Each file should **visualize** and **save** the result of its corresponding task. Document your code where necessary. Your project's file structure should exactly look like this:

```
.
├── data
│   └── ...
├── output
│   └── ...
├── obb.py
├── surface-normals.py
├── curvature.py
├── icp.py
├── svd.py
└── common.py (optional)
```

Use the same file names as shown above. Zip this entire project folder and submit it through Ninova. Do **not** include the `data/` folder in your zip.

If your code uses a different file structure or naming convention, your grade will be penalized significantly. Non-standard submissions create an accumulated burden on the grading process.

Assessment Criteria

Your implementation will constitute 80% of your grade, and will be evaluated based on the following criteria:

- Correctness and completeness of the implementation.
- Readability and clarity of the code.
- Visualization clarity

Also, do not over-engineer the code, the code itself should be self-explanatory.

The project report will constitute 20% of your grade and it should cover the following details:

- An explanation of how you implemented the SVD algorithm from scratch.
- A brief numerical comparison between your SVD implementation and NumPy's SVD.
- A clear and concise explanation of the theory and implementation of each task. You can use pseudocode if necessary.
- Visualizations of the results for each task.
- A discussion of the results. What could be improved? What are the limitations?

About the Usage of AI Tools

ChatGPT, Claude, and many other AI tools are powerful resources that we all have actively used in our researches and projects. However, especially at the beginning of the learning process, these tools can create an illusion of understanding. It might feel like you grasp the concepts when, in reality, you are only scratching the surface. Simply reading and momentarily understanding AI-generated answers does not mean you truly comprehend the material.

To genuinely learn a new topic, you need to engage with the material intentionally and put in focused effort. It is important to embrace the challenges and frustrations that come with problem-solving because this is where meaningful learning happens. AI tools can assist you along the way, but they should complement, not replace, your active participation in the learning process. So, please do not use them right away for your own sake. Only use it to try to understand the underlying concepts.

References

[1] J. Shlens, "A tutorial on principal component analysis," *arXiv preprint arXiv:1404.1100*, 2014. Available at `https://arxiv.org/abs/1404.1100`.

[2] P. Besl and N. D. McKay, "A method for registration of 3-d shapes," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14, no. 2, pp. 239–256, 1992.